DSA MINI PROJECT REPORT

Student Record Management System

Using Singly Linked List in C

Submitted by: Vedant Bhande

Roll Number: 40

Class / Division: B

Department: CSE

College Name: Alard University

Academic Year: 2026–2027

Submitted in partial fulfillment of the requirements for the course in Data Structures and Algorithms.

INDEX

1. Abstract

2. Introduction

3. Problem Statement

4. Objectives

5. Software and Hardware Requirements

6. Data Structure Used

7. Algorithms

8. Time Complexity Analysis

9. Advantages

10. Limitations

11. Future Scope

12. Conclusion

13. References

Appendix A: C Program

Appendix B: Sample Output

1. Abstract

The Student Record Management System is a mini project developed using C and Data Structures and Algorithms (DSA). It allows users to add student records, display all records, search by roll number, and delete records. A singly linked list organizes student information dynamically. Each record contains a roll number, student name, and marks. The project demonstrates linked lists, structures, pointers, dynamic memory allocation, and searching.

2. Introduction

Data Structures and Algorithms help organize data and perform operations efficiently. This project manages student details using a singly linked list. Unlike a fixed-size array, a linked list allows nodes to be allocated dynamically. A menu-driven interface makes the program easy to use.

3. Problem Statement

Managing student information manually can be time-consuming and may lead to errors. A simple computerized system is needed to maintain records and perform basic operations. This project addresses the problem through a menu-driven C application.

4. Objectives

• Understand linked lists.

• Implement a data structure using C.

• Learn structures, pointers, and dynamic memory allocation.

• Add and display student records.

• Search for students using roll numbers.

• Delete records and analyze operation complexity.

5. Software and Hardware Requirements

Software: Windows or Linux, C compiler such as GCC, and a code editor such as Visual Studio Code.

Hardware: Computer or laptop, keyboard, monitor, and approximately 2 GB RAM recommended.

6. Data Structure Used

The project uses a singly linked list. Each node contains a roll number, name, marks, and a pointer to the next node. The head pointer stores the address of the first node. The last node points to NULL.

```
HEAD -> [101, Rahul] -> [102, Priya] -> [103, Amit] -> NULL
```

Linked lists support dynamic insertion and deletion, but searching requires traversing nodes from the head.

7. Algorithms

7.1 Add Student

• Allocate memory for a new node.

• Read roll number, name, and marks.

• Check whether the roll number already exists.

• Reject duplicates; otherwise, append the node at the end.

7.2 Display Students

• Start from the head node.

• If the list is empty, show a message.

• Display each record and move to the next node.

• Stop when NULL is reached.

7.3 Search Student

• Read the roll number to search.

• Traverse the list and compare roll numbers.

• Display a matching record or report that the student was not found.

7.4 Delete Student

• Read the roll number to delete.

• Find the matching node and track the previous node.

• Update the head or previous node's next pointer.

• Free the deleted node's memory.

8. Time Complexity Analysis

Operation	Worst-case time
Add student at end	O(n)
Display records	O(n)
Search student	O(n)
Delete student	O(n)

Here, n is the number of student records. Adding at the end is O(n) because the program traverses the list to find the last node.

9. Advantages

• Easy to understand and use.

• Records can be added dynamically.

• Demonstrates pointers and linked lists.

• No fixed array size is required.

10. Limitations

• Records are lost when the program terminates.

• Searching requires traversal of the list.

• Only basic record management is included.

• No authentication or graphical interface.

11. Future Scope

Future improvements include file handling for permanent storage, updating details, sorting by marks, calculating grades, generating reports, and adding a graphical user interface.

12. Conclusion

The Student Record Management System demonstrates a singly linked list in C. It supports insertion, traversal, searching, and deletion. The project provides practical experience with structures, pointers, dynamic memory allocation, and algorithm complexity.

13. References

• E. Balagurusamy, Programming in ANSI C.

• Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language.

• Class notes and study materials on Data Structures and Algorithms.

• C programming and compiler documentation.

Appendix A: C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Student {
    int roll;
    char name[50];
    float marks;
    struct Student *next;
};
struct Student *head = NULL;

void addStudent() {
    struct Student *newNode, *temp;
    newNode = (struct Student *)malloc(sizeof(struct Student));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    printf("Enter roll number: ");
    scanf("%d", &newNode->roll);
    temp = head;
    while (temp != NULL) {
        if (temp->roll == newNode->roll) {
            printf("Roll number already exists!\n");
            free(newNode);
            return;
        }
        temp = temp->next;
    }
    printf("Enter student name: ");
    scanf(" %49[^\n]", newNode->name);
    printf("Enter marks: ");
    scanf("%f", &newNode->marks);
    newNode->next = NULL;
    if (head == NULL) head = newNode;
    else {
        temp = head;
        while (temp->next != NULL) temp = temp->next;
        temp->next = newNode;
    }
    printf("Student added successfully!\n");
}

void displayStudents() {
    struct Student *temp = head;
    if (temp == NULL) {
        printf("No student records found.\n");
        return;
    }
    printf("\n--- Student Records ---\n");
    while (temp != NULL) {
        printf("Roll Number: %d\n", temp->roll);
        printf("Name: %s\n", temp->name);
        printf("Marks: %.2f\n\n", temp->marks);
        temp = temp->next;
    }
}

void searchStudent() {
    int roll;
    struct Student *temp = head;
    printf("Enter roll number to search: ");
    scanf("%d", &roll);
    while (temp != NULL) {
        if (temp->roll == roll) {
            printf("Student found!\nName: %s\nMarks: %.2f\n",
                   temp->name, temp->marks);
            return;
        }
        temp = temp->next;
    }
    printf("Student not found.\n");
}

void deleteStudent() {
    int roll;
    struct Student *temp = head, *prev = NULL;
    printf("Enter roll number to delete: ");
    scanf("%d", &roll);
    while (temp != NULL && temp->roll != roll) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Student not found.\n");
        return;
    }
    if (prev == NULL) head = temp->next;
    else prev->next = temp->next;
```

```
    free(temp);
    printf("Student deleted successfully!\n");
}

int main() {
    int choice;
    do {
        printf("\n=== Student Record Management ===\n");
        printf("1. Add Student\n2. Display Students\n");
        printf("3. Search Student\n4. Delete Student\n5. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Please enter a number.\n");
            break;
        }
        switch (choice) {
            case 1: addStudent(); break;
            case 2: displayStudents(); break;
            case 3: searchStudent(); break;
            case 4: deleteStudent(); break;
            case 5: printf("Thank you for using the system!\n"); break;
            default: printf("Invalid choice. Try again.\n");
        }
    } while (choice != 5);
    while (head != NULL) {
        struct Student *temp = head;
        head = head->next;
        free(temp);
    }
    return 0;
}
```

Appendix B: Sample Output

```
=== Student Record Management ===
1. Add Student
2. Display Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice: 1
Enter roll number: 101
Enter student name: Rahul
Enter marks: 85
Student added successfully!

Enter your choice: 2
--- Student Records ---
Roll Number: 101
Name: Rahul
Marks: 85.00
```

Note: The program keeps records in memory only. Records are cleared when the application exits.